

AuthContext — Fundamentos

Lógica del Context de autenticación y de los stacks de navegación (GymBro App)

1. ¿Qué problema resuelve el Context de React?

Sin Context, si AuthContext no existiera, para que home_user.js supiera quién está logueado tendrías que: guardar user en el componente más de arriba (App), pasarlo como prop a AppNavigator, pasarlo como prop a cada Stack.Screen, y de ahí a cada pantalla, y de esas a sus hijos...

Eso se llama **prop drilling** y es inviable en una app con muchas pantallas. El Context es el mecanismo de React para decir: “este dato (user) y estas funciones (login, register, logout) están disponibles para **cualquier componente del árbol**, sin pasarlos manualmente por props”.

2. Las tres piezas de AuthContext.js

a) createContext(...) — línea 16

```
const AuthContext = createContext({
  user: null,
  login: () => ({ ok: false, error: '' }),
  register: () => ({ ok: false, error: '' }),
  logout: () => {},
});
```

createContext(valorPorDefecto) crea el **“tubo” (canal)** por el que va a viajar la información. El objeto que le pasás ahí ({ user: null, login: ..., ... }) es solo un **valor de emergencia**: se usa únicamente si algún componente llama a useAuth() sin estar envuelto por un <AuthProvider>. En esta app eso nunca pasa porque AuthProvider envuelve todo desde app/index.tsx, así que ese valor por defecto es más una red de seguridad / documentación de la forma que va a tener el contexto.

Importante para defender: AuthContext en sí mismo **no tiene estado**. Es solo la definición del canal. El estado real vive adentro de AuthProvider.

b) AuthProvider — el que realmente tiene el estado (líneas 25-70)

```
export function AuthProvider({ children }) {
  const [user, setUser] = useState(null); // <- el estado vive ACÁ, con useState normal

  const login = (email, password) => { ... setUser(found) ... };
  const register = (data) => { ... setUser(addUser(data)) ... };
  const logout = () => setUser(null);

  return (
    <AuthContext.Provider value={{ user, login, register, logout }}>
      {children}
    </AuthContext.Provider>
  );
}
```

Este es un componente normal de React, como cualquier pantalla. Tiene su propio useState(null) para user — es exactamente el mismo hook que usás en login.js para email/password. No hay magia ahí.

Lo especial es el return: en vez de devolver un View o un Text, devuelve <AuthContext.Provider value={...}>{children}</AuthContext.Provider>.

- children es todo lo que quede **adentro** de <AuthProvider> en app/index.tsx (o sea, <NavigationContainer><AppNavigator /></NavigationContainer> completo).
- El Provider es un componente especial de React que dice: “todo mi contenido (children), sin importar cuán profundo esté anidado, puede leer el value que le paso acá”.
- value={{ user, login, register, logout }} es un objeto nuevo que se recrea en cada render, con el user actual y las funciones (que además “capturan” el setUser de este AuthProvider por clausura/closure).

c) useAuth() — el hook de lectura (líneas 73-75)

```
export function useAuth() {
  return useContext(AuthContext);
}
```

useContext(AuthContext) es la función de React que dice: “subí por el árbol de componentes hasta encontrar el <AuthContext.Provider> más cercano, y devolveme su value”. useAuth es solo un atajo para no escribir useContext(AuthContext) en cada archivo — hace exactamente lo mismo.

En login.js:

```
const { login } = useAuth();
```

Es equivalente a escribir const { login } = useContext(AuthContext).login — está agarrando la función login que vive dentro del AuthProvider de más arriba en el árbol.

3. El mecanismo que hace que “el navegador cambie de pantalla solo”

Este es el punto que más te conviene tener clarísimo. Encadenemos los eventos:

- 1 Usuario toca “Iniciar Sesión” → handleLogin() en login.js → llama login(email, password) (la función que vino de useAuth(), o sea la de AuthProvider).
- 2 Dentro de esa función, si las credenciales matchean: setUser(found).
- 3 setUser es un **setter de useState**. Llamarlo le dice a React: “el estado de este componente (AuthProvider) cambió, volvé a renderizarlo”.
- 4 React re-renderiza AuthProvider. Ahora user ya no es null, es el objeto del usuario encontrado. Se crea un nuevo value={{ user: found, login, register, logout }} para el Provider.
- 5 Como el value del Provider cambió, **React re-renderiza automáticamente a todos los componentes que consumen ese contexto** con useContext/useAuth — sin que nadie los llame a mano.
- 6 AppNavigator (en app/index.tsx) es justamente uno de esos consumidores.

```
function AppNavigator() {
  const { user } = useAuth(); // <- se vuelve a ejecutar, ahora "user" no es null

  return (
    <Stack.Navigator screenOptions={{ headerShown: false }}>
      {!user ? (
        <> <Stack.Screen name="login" .../> <Stack.Screen name="signup" .../> </>
      ) : user.isTrainer ? (
        <> ...pantallas de entrenador... </>
      ) : (
        <> ...pantallas de usuario común... </>
      )}
    </Stack.Navigator>
  )
}
```

- ```
);
}
```
- Como user ahora es truthy, la condición !user es false → deja de renderizar el <Stack.Screen name="login"> y el <Stack.Screen name="signup">, y en su lugar renderiza **otro grupo de Stack.Screen** (según user.isTrainer).
  - React Navigation ve que el Stack.Navigator tiene un **conjunto de screens totalmente distinto** ahora, y monta el nuevo stack mostrando la **primera screen declarada** de ese grupo (home\_user si no es trainer, o buscar\_usuario\_trainer si es trainer).

Es literalmente eso: no hay ningún navigation.navigate('home\_user') en ningún lado. La navegación es una **consecuencia** de que cambia qué pantallas existen, no una orden explícita.

### Por qué esto es clave conceptualmente

Es el patrón **“pantallas condicionadas por autenticación”**, muy común en apps reales (Instagram, bancos, etc.): el árbol de rutas disponibles depende del estado de sesión, en vez de tener todas las pantallas siempre disponibles y “esconder” unas de otras con lógica de permisos dispersa. Ventajas que podés mencionar si preguntan “¿por qué lo hicieron así?”:

- Un usuario sin sesión **no puede** navegar a home\_user aunque quisiera forzarlo, porque esa ruta directamente no existe en el navigator en ese momento.
- Separa claramente qué rutas son “públicas” (login/signup) de las privadas por rol (trainer vs usuario).

## 4. Los “stacks” — qué es un Stack.Navigator en sí

createNativeStackNavigator() (de @react-navigation/native-stack) crea un componente de navegación tipo **pila** (stack, como una pila de cartas o el call stack): cada pantalla nueva se “apila” arriba de la anterior, y navigation.goBack() o el botón atrás la “desapila”. Es el patrón típico de “voy hacia adelante empujando pantallas, vuelvo atrás sacándolas”.

<Stack.Screen name="login" component={LooginScreen} /> registra una ruta llamada "login" asociada al componente LooginScreen. navigation.navigate('signup') le dice al stack “apilá (o andá a) la pantalla registrada como 'signup'”.

En este proyecto hay **un solo Stack.Navigator**, pero su contenido (qué Stack.Screen existen) cambia dinámicamente según user. No son “3 stacks separados que se intercambian” técnicamente — es un único <Stack.Navigator> cuyos children (los Stack.Screen) se recalculan en cada render de AppNavigator, gracias al if/else de JSX. React Navigation internamente detecta que las rutas disponibles cambiaron y resetea la pila mostrando la primera pantalla del nuevo listado.

## 5. Resumen en una frase para el oral

*“El estado de sesión (user) vive en un useState dentro de AuthProvider, que lo expone a toda la app mediante React Context. Cuando login() llama a setUser, React re-renderiza automáticamente cualquier componente que consuma ese contexto — entre ellos AppNavigator, que decide qué pantallas existen en el Stack.Navigator según si user es null, es entrenador, o es alumno. Por eso el cambio de pantalla tras loguearse es un efecto derivado del cambio de estado, no una llamada explícita a navigation.navigate.”*